Mock Midterm Review
IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

1. Part A: Containers and Version Control
2. Part A: RDBMS and SQL
3. Part B: Reasoning Questions

Part A: Containers and Version Control

Question 1

What distinguishes a distributed version control system from a centralized one?

(a) Only one developer may work on the project at a time.

(b) The repository can only ever be stored on a central server.

(c) Every developer holds a complete copy of the repository, including its history.

(d) Changes are kept locally and can never be shared with anyone else.

Question 1: Answer

Answer: (c)

Every developer holds a complete copy of the repository, including its history.

- The whole point of VC is to ensure that developers keep their working code copies. Syncing with the main truck and sharing with other developers ensures changes arent lost but integrated smoothly.

Question 2

Which of the four Git locations best describes the locations of a file that has been modified and then added with `git add`, but not yet committed?

(a) The working directory.

(b) The staging area.

(c) The local repository.

(d) The remote repository.

Question 2: Answer

Answer: (b)

The staging area.

- Unless committed and pushed, the file is staged for commit and sits in the staging area. The file could be a part of the working directory but is different from a file in the working directory that has not been committed and untracked by Git.

Question 3

Which of the following is true for a container:

(a) Always requires installation of libraries and packages necessary to run the underlying code.

(b) Always requires access to underlying folders and directories.

(c) Cannot be moved between different physical machines.

(d) Provides virtualization, packaging only the dependencies an application needs.

Question 3: Answer

Answer: (d)

Provides virtualization, packaging only the dependencies an application needs.

- Containers ensure that libraries and dependencies are pre-packaged. Virtualized runtime makes code execuction seamless on user end.

Question 4

What is the purpose of a container registry?

(a) To store and share container images so that others can pull and run them.

(b) To record which containers are currently running on a host machine.

(c) To assign port numbers to running containers automatically.

(d) To collect error logs produced by containers for later debugging.

Question 4: Answer

Answer: (a)

To store and share container images so that others can pull and run them.

- Registry like Dockerhub make sure that container images can be shared and accessed by the wider public. Access control is also enabled on an opt-in basis.

Question 5

Adding the `:ro` tag to a volume mount ensures that:

(a) The container may both read from and write to the mounted location.

(b) The container may read from the mounted location but cannot write to it.

(c) The host machine loses the ability to read the mounted files.

(d) The mounted data is deleted as soon as the container stops.

Question 5: Answer

Answer: (b)

The container may read from the mounted location but cannot write to it.

- This is a read-only tag which means code inside the container can read but not write to the location on the host computer. This is used to gate access to underlying content.

Part A: RDBMS and SQL

Question 6

In RDBMS design, an identifier for an entity must:

(a) Always consist of exactly one attribute.

(b) Describe a physical characteristic of the entity.

(c) Take on a unique value for each instance of that entity.

(d) Be stored using a character data type.

Question 6: Answer

Answer: (c)

Take on a unique value for each instance of that entity.

- Identifiers must always be chosen to help uniquely identify entities themselves. This means having a unique identifier for each row of a rtable for example.

Question 7

Cardinality must be specified in both directions of a relationship because:

(a) Every relationship is bidirectional.

(b) The DBMS requires two foreign keys for every relationship.

(c) All relationships are ultimately many-to-many.

(d) A relationship always involves three or more entities.

Question 7: Answer

Answer: (a)

Every relationship is bidirectional.

- Even though in some cases, we have one-to-one relationships between entities, cardinality helps ensure that these relationships (many-to-on, one-to-one) are explictly specified.

Question 8

An associative entity is introduced in order to resolve a relationship that is:

(a) One-to-one.

(b) One-to-many.

(c) Recursive.

(d) Many-to-many.

Question 8: Answer

Answer: (d)

Many-to-many.

- Associative entities model many-to-many relationships. As an example, in the customer order example this ensure that there can be many customers who place many orders.

Question 9

A relationship between different instances of the same entity, such as a course and the course that is its prerequisite, is called:

(a) A binary relationship.

(b) A ternary relationship.

(c) A recursive, or unary, relationship.

(d) An associative relationship.

Question 9: Answer

Answer: (c)

A recursive, or unary, relationship.

- The entities in question are of the same type. Therefore, this is a recursive relationship.

Question 10

Which statement about a relation is correct?

(a) Rows occupy fixed positions and must be viewed in that order.

(b) A single row may hold multiple values at its intersection with a column.

(c) No two rows in a relation may be duplicates of one another.

(d) The same column name may appear more than once in a table.

Question 10: Answer

Answer: (c)

No two rows in a relation may be duplicates of one another.

- Two rows having the same content must necessarily be distinguished based on a unique identifier i.e. the key.

Question 11

The referential integrity constraint states that:

(a) Every primary key value must be numeric.

(b) Every non-null foreign key value must match an existing primary key value.

(c) Every table in a database must contain at least one foreign key.

(d) Foreign key values must be unique within the table that holds them.

Question 11: Answer

Answer: (b)

Every non-null foreign key value must match an existing primary key value.

- Foreign keys must not be dangling and must point to a primary key in another table. If not, then the DBMS queries will start failing and integrity is compromised.

Question 12

An employee table records each employee's several children in a single repeating group. The correct way to handle this is to:

(a) Add a fixed number of columns, one for each child.

(b) Store the children's names separated by commas within one column.

(c) Leave the design unchanged, since repeating groups are permitted in first normal form.

(d) Create a second table holding one row per child, linked back by the employee number.

Question 12: Answer

Answer: (d)

Create a second table holding one row per child, linked back by the employee number.

- Creating the second table linked with a foreign and primary key pair would help make information storage streamlined. This vaoids duplication and ensures queries are efficient.

Question 13

A query that needs columns drawn from three different tables requires:

(a) Two join conditions, because a join works on two tables at a time.

(b) One join condition, which the DBMS applies across all three tables.

(c) Three join conditions, one for each table named in the query.

(d) No join conditions, since the DBMS infers them from the foreign keys.

Question 13: Answer

Answer: (a)

Two join conditions, because a join works on two tables at a time.

- LeftTable $\bigcap$ MiddleTable $\bigcap$ RightTable. Each intersection is a join operation, hence two of these.

Question 14

The `PRIMARY KEY` constraint is equivalent to which combination of constraints?

(a) `NOT NULL` together with `CHECK`.

(b) `UNIQUE` together with `DEFAULT`.

(c) `NOT NULL` together with `UNIQUE`.

(d) `FOREIGN KEY` together with `UNIQUE`.

Question 14: Answer

Answer: (c)

`NOT NULL` together with `UNIQUE`.

- Primary keys cannot be null and need to be unique. Declaring a column as primary key is tantamount to making this happen.

Question 15

Why do most DBMSs leave duplicate rows in a result table unless `DISTINCT` is explicitly requested?

(a) Duplicate rows are required in order to maintain referential integrity.

(b) Removing duplicates can be computationally expensive.

(c) The SQL standard forbids a DBMS from removing them automatically.

(d) `DISTINCT` can only be applied to numeric columns.

Question 15: Answer

Answer: (b)

Removing duplicates can be computationally expensive.

- The SQL engine does not explicitly assume user intent especially when it comes to query results (such as removal) which could be very expensive.

Question 16

In the clause `ORDER BY zip_postcode DESC, last_name ASC`, the role of `last_name` is to:

(a) Replace the sort on `zip_postcode` entirely.

(b) Sort rows within each group of rows sharing the same `zip_postcode`.

(c) Remove rows that have no value for `last_name`.

(d) Decide which columns appear in the result table.

Question 16: Answer

Answer: (b)

Sort rows within each group of rows sharing the same `zip_postcode`.

- The query aligns entries by descending order of zip. Within each zip, it will order names in alphabetical order of last name.

Question 17

A `DROP TABLE` statement will be refused if:

(a) The table currently contains one or more rows.

(b) The table has a primary key declared on it.

(c) The user has not issued a `SELECT DISTINCT` first.

(d) Another table's foreign key still references it.

Question 17: Answer

Answer: (d)

Another table's foreign key still references it.

- By default the MySQL querying will fail to proceed if we still have another tables foreign key references. This is done to make sure that there is referential integrity.

Question 18

A question such as "which vendors have never sent us a shipment" cannot be answered using an inner join because:

(a) Rows having no matching row in the other table are dropped from the result.

(b) An inner join cannot be combined with a `WHERE` clause.

(c) An inner join requires the tables to share a concatenated key.

(d) MySQL does not implement inner joins.

Question 18: Answer

Answer: (a)

Rows having no matching row in the other table are dropped from the result.

- Inner Joins require validity of entries from both tables. The nature of the query is such that a vendor satisfying its constraints will have no corresponding entires in the shipment table.

Question 19

A foreign key declared with `ON DELETE SET NULL` requires that:

(a) The parent table has no primary key of its own.

(b) The child table is empty at the time of the delete.

(c) The foreign key column permits null values.

(d) The delete statement is issued without a `WHERE` clause.

Question 19: Answer

Answer: (c)

The foreign key column permits null values.

- Foreign keys if not allowed to be NULL will make this query fail since it cannot fulfill the user request of deleting and setting keys to NULL as a policy

Question 20

An entity whose primary key consists of a single attribute:

(a) Cannot possibly be in first normal form.

(b) Is automatically in second normal form, because a partial dependency is impossible.

(c) Must contain at least one repeating group.

(d) Automatically satisfies third normal form as well.

Question 20: Answer

Answer: (b)

Is automatically in second normal form, because a partial dependency is impossible.

- One attribute means that partial dependency scenario does not arise. Therefore, such entity modeling is already in second normal form.

Part B: Reasoning Questions

Part B, Question 1

Two companies ask you to model the association between an Employee and a Store. At the first company, every employee is permanently assigned to exactly one store. At the second, employees rotate and may work at several different stores in the same week. Explain why these two situations do not produce the same database model, and describe what structural difference appears in the second one.

Part B, Question 1: Answer

- In the first case an employee belongs to exactly one store while a store has many employees, which is a one-to-many relationship. It is captured by ensuring that the store's primary key shows up in the Employee entity as a foreign key, and no new entity is needed.
- In the second company an employee works at many stores and a store has many employees, which is a many-to-many relationship.
- A foreign key cannot represent this in either direction, because a single foreign key column can hold only one value, so an associative (composite) entity is required that can uniquely identify an (Employee and Store) combination.
- Its primary key therefore, is the combination of the employee identifier and the store identifier, and each row records one employee working at one store.
- Asa result, correct design depends on the business rules, not on purely the entity names themselves. The same two entities produce two different models because the two businesses operate differently.

Part B, Question 2

Consider a schema similar to our lecture examples (Lecture 8) that declares its foreign keys with `ON DELETE RESTRICT`, and another which asks for `ON DELETE CASCADE`. A warehouse manager now deletes a customer who has fifteen orders recorded against them. Explain what happens in each of the two schemas, and argue which behavior is the safer choice for a production order-entry system.

Part B, Question 2: Answer

- Under `ON DELETE RESTRICT`, the DBMS refuses the delete outright because child rows in the orders table still reference that customer. The statement fails with aforeign key constraint error.
- The customer remains and does not get deleted, and all fifteen orders remain intact. To address this the user must deal with the orders first and delete them before proceeding.
- Under `ON DELETE CASCADE`, the delete succeeds and all fifteen orders are removed along with the customer, even though the `DELETE` statement never mentioned the orders table. If the orders table is itself a parent to order line rows, that deletion propagates further.
- For a production order-entry system, `RESTRICT` is naturally safer. Orders are financial and historical records and should not be tampered with under mistakes or erroneous query results.
- `CASCADE` is appropriate where child rows have no meaning without the parent.

Part B, Question 3

Suppose we want a list of every customer together with any orders they have placed, *including* customers who have never placed an order at all. Suppose that we write the query below, but the result contains no customers without orders. Explain why the query does not do what we intended to achieve.

```
SELECT c.Customer_FirstName, c.Customer_LastName, co.Order_Num
FROM customer c
  LEFT JOIN customerorder co
  ON c.Customer_Num = co.Customer_Num
WHERE co.Order_Date >= '2021-10-01';
```

Part B, Question 3: Answer

- The `LEFT JOIN` does its job and produces a row for every customer, with `NULL` in all the `customerorder` columns for those who have no orders.
- The `WHERE` clause is then applied to that intermediate result, and it tests `co.Order_Date >= '2021-10-01'`.
- For a customer with no orders, `co.Order_Date` is `NULL`, and a comparison against `NULL` is never true.
- Those rows are therefore filtered out after the join, which leaves exactly the rows an inner join would have produced.
- Under Left Joins, a condition on the right-hand table placed in the `WHERE` clause defeats an outer join. To keep the unmatched rows, the condition has to move into the `ON` clause so it is part of the matching rather than a filter applied afterwards.

Part B, Question 4

A table holds the columns `EmployeeID`, `EmployeeName`, `DeptCode`, and `DeptManager`. `EmployeeID` is the primary key, `DeptCode` depends on `EmployeeID`, and `DeptManager` depends on `DeptCode`. Identify the normal form this design violates and explain the practical problem it will cause when a department changes its manager.

Part B, Question 4: Answer

- This design specifically violates third normal form or 3NF. `DeptManager` is a non-key attribute that depends on `DeptCode`, which is itself a non-key attribute, so `DeptManager` depends on the primary key only transitively, through `DeptCode`.
- The relation is in 2NF, since the primary key is a single attribute and no partial dependency is possible.
- The practical problem is that the manager of a department is stored once for every employee in that department rather than once in total. When the department appoints a new manager, every row for every employee of that department must be updated together.
- If any row is missed, the table now reports two different managers for the same department and there is nothing in the design that can say which is correct.
- The fix is to move `DeptCode` and `DeptManager` into a separate Department relation keyed on `DeptCode`, leaving `DeptCode` in the employee table as a foreign key.

Part B, Question 5

Explain why volume mounting is necessary before a program running inside a container can work with data files that live on the host machine, and describe what changes when that mount is declared read-only.

Part B, Question 5: Answer

- A container has its own isolated filesystem, built from its image, and nothing on the host is visible inside it by default.
- A program running in the container therefore cannot open a file sitting on the host no matter what path it uses.
- Volume mounting makes a host path available at a chosen path inside the container, so the same bytes are reachable from both sides, and the container works with the live files rather than a copy baked into the image.
- Adding the `:ro` tag makes the mount read-only. The container can still read the files, but any attempt to write to, modify, or delete them is refused.
- This is the appropriate choice whenever the container only needs to consume the data, since it removes the possibility that a bug in the containerized program damages the original files on the host.